R2WX [2024] — An Engineering Exploration

About this examination

This document presents the results of a structured investigation conducted between the author of R2WX and two artificial analytical agents, ChatGPT and Gemini, operating under deliberately different roles.

The investigation did not begin with the eight findings presented below.

Its first level established a defensible factual basis through author interrogation against original implementation material. Claims, recollections and intended behavior were repeatedly compared with concrete artifacts. Findings were progressively challenged, qualified, rejected where necessary, and documented when sufficient agreement between implementation evidence and author knowledge had been established.

A second level reopened that basis through a broader reconstruction of R2WX as a whole system.

Gemini was assigned the repository-facing role: blind excavation of the implementation, independent evidence gathering, and development of candidate interpretations from what could actually be found. ChatGPT occupied a deliberately resistant analytical position, repeatedly withholding broader interpretations when the available evidence did not yet justify them. The author supplied the implementation history, engineering intent and accumulated reasoning that source code alone could not recover, while also challenging interpretations that either overstated the artifact or stopped before apparently unrelated implementation decisions could be reconciled.

These roles were deliberately different. Their purpose was not agreement between three participants, but continued interrogation from positions with different access to the object being examined.

The investigation repeatedly reached conclusions that appeared sufficient and then continued when additional evidence changed, restricted, strengthened or reorganized them. The structure that eventually emerged—eight connected pressure points across the full stack—was therefore a result of the investigation rather than its starting framework.

The complete analytical model, its successive levels, assigned roles, evidence distinctions and representative prompts are documented separately in [Investigation Design →].

What R2WX is

R2WX is a runnable personal full-stack R&D artifact from 2024.

It uses the familiar CRUD/REST application model as an experimental ground for examining recurring engineering pressures across a full-stack system: where application knowledge belongs, when repeated knowledge becomes repeated procedure, how reusable machinery accommodates specialization, which capabilities become exposed, how authority changes across boundaries, which layers should interpret transported data, and where consolidation should stop.

Its implementation crosses the complete application path: browser-facing Svelte/SvelteKit components and forms, a SvelteKit BFF, a NestJS core API, shared application schemes and reusable backend machinery, Prisma persistence, MySQL and filesystem storage, composed as a runnable Docker environment.

CRUD is therefore not the research question of R2WX. It provides a deliberately familiar operational environment in which these questions can be exercised together.

What the investigation found

The examination produced eight connected findings.

1. Shared knowledge can reduce repeated implementation without requiring every consumer to surrender its own responsibility.

R2WX allows common application definitions to be consumed across frontend and backend machinery while keeping decisions that belong specifically to those consumers local.

In practice: a field constraint does not need to be independently rediscovered and reimplemented everywhere it is required, while the frontend can still use that knowledge to construct interaction and the backend can independently use it to determine acceptance.

2. Sharing knowledge and sharing ownership are different decisions.

The same structural knowledge can serve several parts of the application without collapsing their distinct concerns into one universal configuration.

In practice: frontend representation information and backend routing/API information remain separately owned even where both operate around the same entity definition.

3. Once recurring knowledge produces recurring work, the procedure itself can become shared infrastructure.

R2WX does not stop at consolidating definitions. Repeated operations that can legitimately follow those definitions are moved into reusable machinery.

In practice: entities do not require independent implementations of the same validation and standard CRUD procedures every time another entity is introduced.

4. Shared procedure does not require exceptional behavior to remain inside the generic path.

R2WX provides common lifecycle machinery while retaining explicit points at which an application can compose with, depart from, or replace generic behavior.

In practice: a specialized submission or service operation can perform application-specific work without requiring the common form, request or persistence lifecycle to be rebuilt around it.

5. Reusable capability and exposed capability are separate decisions.

Infrastructure may know how to perform an operation without every consumer of that infrastructure automatically making the operation available.

In practice: inheriting a controller that contains standard CRUD capabilities does not by itself publish all of those endpoints. Exposure is explicitly admitted by the entity configuration and enforced at the receiving boundary.

6. One request does not require one identity to remain authoritative across every boundary it crosses.

R2WX separates the human authorization domain from the internal service authorization domain.

In practice: the BFF evaluates the human session and role, while the core API receives and authorizes a distinct service identity for the internal crossing. Human identity does not simply propagate through the entire request path as the core service credential.

7. Transporting data across a boundary does not necessarily make that boundary responsible for interpreting it.

Intermediate infrastructure can preserve a payload's passage without acquiring ownership of its application meaning.

In practice: multipart uploads can pass through the SvelteKit BFF as an unparsed request stream rather than being parsed into files and fields there and reconstructed before being sent to the NestJS API.

8. Consolidation can stop deliberately where independent authority still matters.

R2WX does not attempt to make its application schemes the sole authority over the entire stack. Persistence retains an independently maintained model and its own final constraints.

In practice: application schemes govern behavior within their runtime jurisdiction, but data that reaches persistence must still satisfy the Prisma/database model. Repetition at that boundary is accepted where eliminating it would also eliminate independently owned persistence authority.

What those findings mean operationally

For an entity that remains within the shared path, the standard CRUD lifecycle can be brought into operation primarily through setup rather than repeated implementation.

That distinction matters.

R2WX does not generate an application or eliminate entity instantiation. Entities still require their appropriate schemes, configuration, routes, wiring and application-specific decisions. What changes is how much recurring behavior must be independently implemented after that setup exists.

The working design heuristic behind the artifact is approximately 75/25: for a conventional application whose requirements largely fit the shared lifecycle, roughly 75% of recurring behavior is expected to remain within shared infrastructure, leaving roughly 25% as application-specific implementation and extension.

The ratio is not a measured constant. Increasing specialization naturally changes it.

Nor is the 25% treated as code that must live permanently outside the shared system. Application-specific procedure can leave the generic path where necessary, perform the work that makes it exceptional, and—where the receiving contract permits—rejoin shared machinery before reaching the next enforced boundary.

The objective is therefore not complete automation.

It is to allow recurring knowledge and procedure to become infrastructure without making that infrastructure a prison for the behavior that does not recur.

Eight pressure points

The examination follows the artifact through eight places where these decisions become visible.

01 — Repeated knowledge

A single application change begins requiring corresponding changes across multiple layers.

→ When One Change Started Requiring Too Many Changes

02 — Shared knowledge, different responsibilities

Consolidating common definitions begins to blur decisions that still belong to different consumers.

→ When Sharing Knowledge Started Blurring Ownership

03 — Repeated procedure

Once common knowledge has been established, the same operational work begins appearing around it.

→ When Shared Knowledge Started Repeating the Same Work

04 — Specialization pressure

Generic machinery works until an application legitimately needs behavior that does not fit its preferred path.

→ When Automation Started Fighting the Application

05 — Capability exposure

Reusable infrastructure accumulates operations that an individual consumer may not intend to expose.

→ When Inheritance Started Giving Too Much

06 — Authority crossing

A request originating from a human crosses an application boundary into another authorization domain.

→ When One User Request Became Two Different Identities

07 — Boundary ownership

An intermediate layer must transport increasingly varied data without necessarily becoming responsible for understanding it.

→ When the Boundary Started Doing Too Much Work

08 — Persistence authority

A system that has repeatedly consolidated knowledge and procedure reaches a boundary where another representation still claims independent authority.

→ When Consolidation Stopped at Persistence

The examination begins with the smallest-looking pressure of the eight:

one field changes—and several parts of the application suddenly need to know.

WHEN ONE CHANGE STARTED REQUIRING TOO MANY CHANGES

schema-driven · schema-validation · schema-driven-forms · configuration-driven · separation-of-concerns

THE BATTLE

A field changes.

Its maximum length becomes 160 instead of 80. A field becomes required. An accepted value changes. A new property is added to an entity.

The change itself is usually simple.

Then the same change has to be made everywhere else that carries its own version of that knowledge.

The form definition. Frontend constraints. Backend validation. DTOs or operation-specific representations. Sometimes persistence.

None of these is necessarily difficult work.

But across fields, entities and operations, repeatedly declaring and synchronizing the same application facts consumes considerable development time and attention. Every additional declaration is also another place that can remain unchanged when the others move forward.

I had encountered this kind of work enough times to stop treating all of it as necessary application work.

That was one of the battles R2WX chose to move into infrastructure.

Not by eliminating frontend and backend responsibilities.

Not by making one runtime subordinate to the other.

The target was narrower:

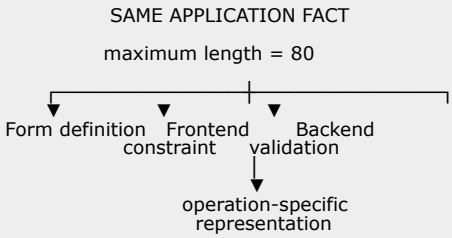
If several parts of the application need the same fact, why should the developer keep declaring that fact independently for each of them?

The first target was the knowledge itself

Consider a simple constraint:

maximum length = 80

When independently represented, one application fact can become several maintenance points:



Now change 80 to 160.

The problem isn't writing 160.

The problem is that the developer has become responsible for knowing every place where 80 independently means the same thing.

```
80 → 160

Where else does 80 live?
        │
        ├── form?
        ├── frontend validation?
        ├── backend validation?
        ├── DTO?
        └── another operation?
```

Miss one and the application can begin disagreeing with itself.

That made the recurring problem larger than duplicated code:

> **Repeated knowledge creates repeated maintenance.**
> **Repeated maintenance creates opportunities for drift.**

R2WX therefore attacks the repetition one level earlier.

It shares the knowledge from which those behaviors can be produced.

Shared application schemes

R2WX places recurring application facts into shared schemes.

A scheme describes the application knowledge associated with an entity and its fields: types, literals, constraints and other rules that multiple consumers may need.

Implementation evidence — the shared application knowledge

```
// modules/src/schemes/samx/index.js

export const samxScheme = () => ({
    id:   { ...T.NUMB, required: false },
    name: { ...T.TEXT, maxLength: 160, required: true },

    description: ...,
    status: ...,
    ...
});
```

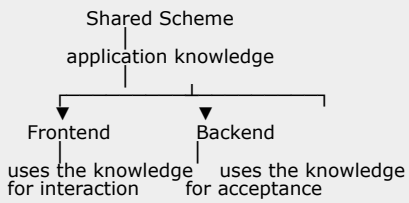
Conceptually:

```
const scheme = () => ({
    field: {
        type: ...,
        required: ...,
        min: ...,
        max: ...,
        // ...
    }
});
```

Instead of:

```
Frontend fact ── maintained independently
Backend fact  ── maintained independently
Form fact     ── maintained independently
```

R2WX moves toward:



The change is small to describe but significant in daily development.

When a shared rule changes, the developer changes the knowledge rather than hunting through independent copies of that knowledge merely because different runtimes consume it.

The first thing R2WX tried to stop repeating was not code. It was application knowledge.

That did not mean merging frontend and backend

This created another question immediately.

If the knowledge is shared, should everything surrounding it also become shared?

R2WX's answer was no.

The frontend and backend consume common application knowledge, but they do different work with it.

On the frontend, ctx.js provides frontend-specific context:

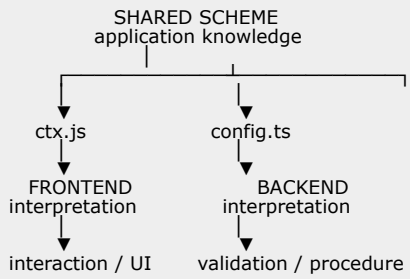
```
{
   entity: ...,
   typ: ...,
   lit: ...
}
```

On the backend, config.ts connects application knowledge to server-specific concerns:

```
export default () => ({
   path: 'user',
   entity: 'APP_USER',
   type: userAuthScheme(),
   api: {...}
})
```

At this point, api does not need to mean anything more than backend-owned configuration. Its exposure role becomes important later.

For now, the important shape is:



The scheme carries what can legitimately be shared.

Each runtime retains the decisions that belong to it.

That is where a familiar architectural term becomes useful:

separation-of-concerns

But the term came after the problem.

The development pressure was much simpler:

Stop declaring the same facts repeatedly without turning frontend and backend into the same responsibility.

The shared knowledge started doing work

Once those declarations existed in a common form, they could become more than descriptions.

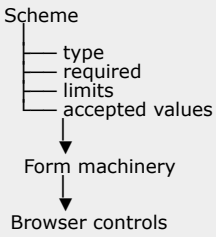
The frontend could interpret them through shared form machinery:

Implementation evidence — the same knowledge reaches the frontend

```
// R2wxInput.svelte

let input = $derived(
    filterInputAttrs($active.form[id] || { type: 'text' })
);

<input {...input} ... />
```



The backend could interpret the same application knowledge through its validation machinery:

Implementation evidence — the backend consumes the same rules

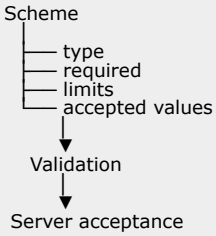
```
// BaseController.ts

@Post()
async create(@Body() data, @Req() req: RequestAuthz) {
    this._assureEndpointAccessible(HTTP.POST, PATH._, req.userAccess);
    this._assureDataValid(data, VALID.STRICT);

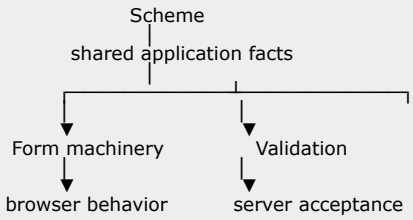
    return { data: await this.service.create(data) };
}
```

```
// Validation.ts

const TMAP = {
    required:  CHECK.EXIS,
    maxLength: CHECK.MAXL,
    ...
};
```



Put together:



This is where labels such as schema-driven-forms and schema-validation become concrete descriptions of what R2WX is actually doing.

The frontend still owns interaction.

The backend still owns acceptance.

The knowledge is shared. Its interpretation and authority remain local.

DTO repetition became part of the same question

Backend DTOs can be useful.

But inside the recurring CRUD path R2WX was exploring, they also raised a practical question.

If the application already knows that a field is:

```
string
required
maximum 160
```

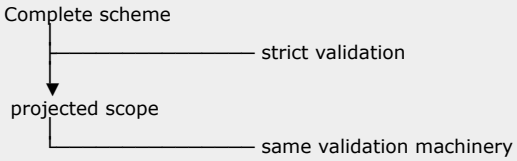
and the backend validation machinery can consume that knowledge directly, reproducing the same constraints through another DTO adds another representation that must remain synchronized.

As entities and operations accumulate, those representations accumulate with them.

R2WX deliberately moved that DTO proliferation out of its CRUD path.

Instead, backend validation consumes the scheme and shared validation machinery directly.

For partial operations, R2WX does not introduce an entirely separate validation system either. It changes the scope being validated and reuses the existing procedure.



The decision wasn't:

DTOs are wrong.

It was much more specific:

R2WX would not require another declaration of application knowledge merely because its shared CRUD machinery had crossed into NestJS.

The framework remained a tool used by the application.

It did not get to decide that the application must reproduce knowledge simply to satisfy one conventional representation of the framework.

Sharing definitions introduced a different danger

Removing repeated declarations did not mean every consumer should manipulate one global mutable object.

Frontend and backend processing may need to enrich or modify the representation they are currently working with.

If every consumer operated on the same object, sharing knowledge could create another form of coupling.

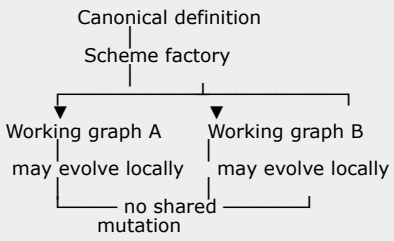
R2WX therefore exposes schemes through factories that materialize independent working graphs:

Implementation evidence — independent working representations

```
// modules/src/schemes/index.js

export const samxScheme = () =>
    structuredClone(samxSchemeCore());
```

So:



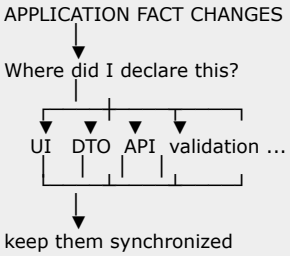
That produced an important distinction:

> **Definitions can be shared without sharing every working representation of them.**

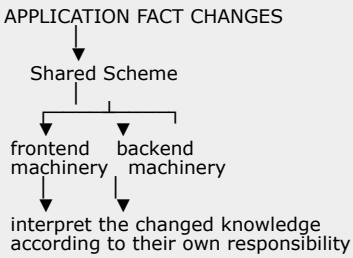
R2WX could therefore consolidate application knowledge without requiring every consumer to share mutable runtime state.

What actually changed for the developer?

Without the shared structure:



With the R2WX direction:



That is the development work R2WX was trying to absorb.

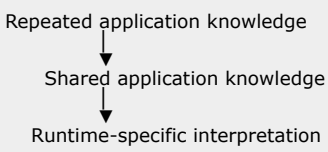
Not every difference between frontend and backend.

Not every application decision.

Not persistence authority.

The recurring work of independently redeclaring knowledge that the application already possessed.

And that gives us the first piece of R2WX's larger path:



The next question follows naturally.

Once the same knowledge could be interpreted repeatedly by the system, how much of the procedure surrounding that knowledge could also stop being repeatedly implemented by the developer?

That is where shared knowledge starts becoming shared procedure.

WHEN SHARING KNOWLEDGE STARTED BLURRING OWNERSHIP

separation-of-concerns · configuration-driven · decoupling · frontend-architecture · backend-architecture

THE BATTLE

Frontend and backend often need to agree on the same application facts.

Keeping separate copies means keeping them synchronized.

But solving that problem by continuously moving more knowledge into one shared definition creates the opposite pressure:

if both sides use it, should both sides own it?

R2WX had already chosen to share application schemes rather than repeatedly maintain equivalent field knowledge.

That solved one synchronization problem.

It did not make frontend and backend the same system.

This distinction matters even though the public R2WX infrastructure runs together through Docker Compose. Its frontend and backend were developed as independently responsible sides of an application—as if they could live at a distance, maintained and operated across their own boundaries.

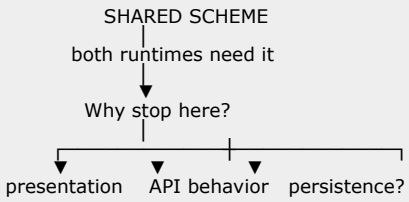
That earlier reality is important to the design.

Two sides can share knowledge.

They still need to know where their agreement ends.

The tempting solution is to keep centralizing

Once a shared scheme works, centralization becomes attractive.



That looks efficient.

But these are no longer necessarily different copies of the same truth.

They may be different truths belonging to different responsibilities.

A frontend needs to know how application knowledge participates in interaction.

A backend needs to know how that knowledge participates in an API and server procedure.

The fact that both sides begin with the same entity does not make everything they know about that entity shareable.

R2WX needed synchronization without turning synchronization into shared ownership.

ctx.js and config.ts draw that line

They can look surprisingly similar from a distance.

Both sit near entity configuration.

Both connect shared application knowledge to runtime behavior.

Both help machinery understand what it is dealing with.

But they do not contain two copies of the same configuration.

They hold different truths.

Implementation evidence — same scheme, different runtime truths

```
// frontend: ctx.js

export default () => ({
    entity: 'sam',
    typ: samScheme(),
    lit: samLit
});
```

```
// backend: config.ts

export default () => ({
    path: 'sam',
    entity: 'TOOL_SAM',
    type: samScheme(),

    api: {
        GET: { [PATH._]: [ACCESS.GLOB], [PATH.ID]: [ACCESS.GLOB] },
        POST: ...,
        ...
    }
});
```

On the frontend:

```
{
    entity: ...,
    typ: ...,
    lit: ...
}
```

ctx.js provides the frontend with the context it needs to interpret the application representation.

On the backend:

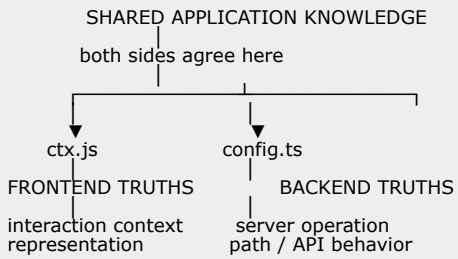
```
export default () => ({
    path: 'user',
    entity: 'APP_USER',
    type: userAuthScheme(),
    api: {...}
})
```

config.ts connects that representation to backend concerns such as type interpretation, path and API configuration.

The important contrast is therefore not:

Frontend config vs Backend config

It is:



Neither runtime needs to own the other's interpretation merely because both consume the same scheme.

That is separation-of-concerns in a very concrete R2WX form.

This distinction came from treating the two sides as capable of living apart

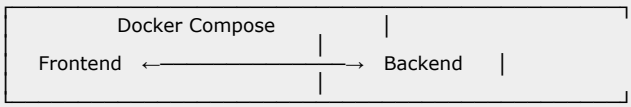
---

Put everything in one Compose stack and some boundaries can begin to look unnecessarily strict.

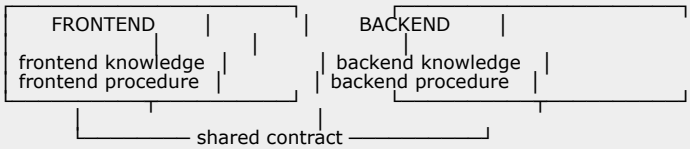
Why maintain a clear frontend/backend separation when both repositories, services or containers are sitting beside each other?

Because physical proximity is not the same thing as responsibility.

The design treats this:



as capable of behaving conceptually like this:



That changes what “shared” means.

Shared cannot simply mean:

put everything both sides know into the same object.

It has to mean:

identify the knowledge over which the two sides genuinely need agreement, while allowing each side to remain independently responsible for what happens beyond that agreement.

This also explains why some R2WX boundaries can appear stricter than necessary when the artifact is inspected only as a locally composed application.

The distance is architectural rather than geographical.

Even frontend knowledge has boundaries

Implementation evidence — structure and literals remain separate

```
// structural scheme

export const samScheme = () => ({
   name:   { ...T.TEXT },
   number: { ...T.NUMB, min: 1, max: 15 },
   ...
});
```

```
// lit.js

export const samLit = {
   name:   'TEXT',
   number: 'NUMBER',
   ...
};
```

There is a smaller decision inside the frontend that helps expose the same principle.

ctx.js autonomously supports frontend interpretation, but not every piece of frontend-facing knowledge was simply absorbed into it.

Literals were kept separately in lit.js.

At first glance, putting them directly into frontend context could look simpler:

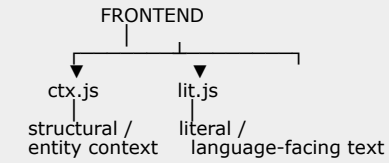
```
ctx.js
 ├── entity
 ├── types
 └── all display literals
```

But literals carry a characteristic that the structural context does not:

their representation can depend on language.

So R2WX keeps a distinction between structural frontend context and literal/display knowledge rather than making the context itself the permanent owner of both.

Conceptually:



There is an important boundary to state here.

The current R2WX artifact should not be presented as demonstrating a complete multilingual delivery system unless the repository actually provides that mechanism.

Its current behavior may effectively serve a single language.

What the structure supports us saying is narrower:

Literal knowledge was kept separate from frontend structural context in a way that preserves language-dependent text as a distinct concern.

The separation is visible.

A complete multilingual mechanism should not be invented from that separation alone.

Implementation evidence — the two runtime contexts feed different machinery

```
// +page.server.js

const { entity, typ, lit } = ctx();

const form = packEntityForm(typ, lit, null, {
    active: { hidden: true },
    id: { hidden: true }
});
```

```
// Sam.controller.ts

const { path, type, api } = config();

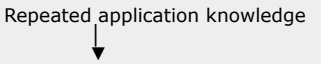
@Controller(path)
export class SamController extends BaseController {
    constructor(service: SamService, svMulter: MulterService) {
        super(service, type, api, svMulter);
    }
}
```

And that is actually more interesting than pretending the capability is complete: the boundary exists even where the larger capability it could support is not fully exercised by this artifact.

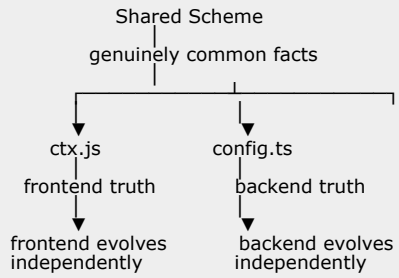
Sharing stopped when the truths stopped being the same

This gives R2WX a more precise answer to the question Section 1 left open.

Section 1 established:



But consolidation does not continue indefinitely:



The stopping criterion isn't simply:

“Can this information technically be put in the shared object?”

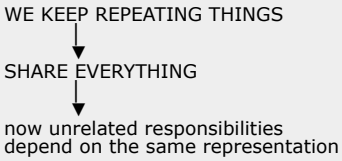
It is closer to:

Does this still represent the same application truth, or has responsibility now crossed into a runtime that owns a different decision?

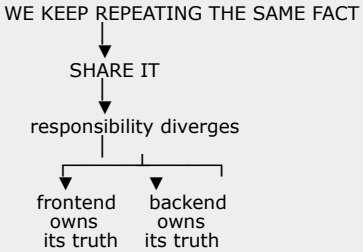
That distinction lets R2WX reduce synchronization work without trying to erase the systems that have to synchronize.

What changed for the developer?

Without the distinction, solving duplication can become another form of coupling:



R2WX instead moves toward:



That is a different kind of relief for the developer.

The system no longer asks for independent copies merely because frontend and backend are separate.

But it also does not require one side to understand or carry the other's internal decisions merely because some knowledge is shared.

Share agreement. Preserve ownership.

And this gives us the next pressure naturally.

Once each runtime could interpret common knowledge while retaining its own responsibility, another kind of repetition became visible:

the same interpretations were repeatedly leading to the same procedures.

Forms still had to behave.

Requests still had to be validated.

CRUD operations still had to execute.

So the next question became:

If knowledge can stop being repeatedly declared, can the procedures that repeatedly consume that knowledge stop being repeatedly implemented too?

That is where R2WX moves from shared knowledge into shared procedure.

WHEN SHARED KNOWLEDGE STARTED REPEATING THE SAME WORK

reusable-infrastructure · schema-driven · schema-validation · form-lifecycle · crud-application · separation-of-concerns

THE BATTLE

Removing duplicate definitions does not remove duplicate implementation.

The application may now know that a field is required, that another is boolean, that one accepts a bounded string and another represents a file.

But somebody still has to turn that knowledge into behavior.

Forms still need to be assembled. Values still need to be collected and normalized. Requests still need to be validated. CRUD operations still need to reach the appropriate service. Responses still need to come back.

If every entity performs those steps independently, shared knowledge has solved only half of the repetition.

This was the next kind of work R2WX chose to absorb.

The question was no longer only:

Where should this application fact be declared?

It became:

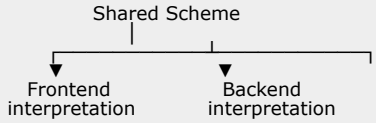
If the system already understands the fact, how much of the recurring work around that fact can the system perform too?

Knowing that a field is required is not enough

Section 1 moved recurring application knowledge into shared schemes.

Section 2 kept the interpretation of that knowledge with the runtime that owns it.

That gives us:



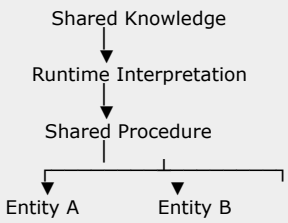
But without reusable procedure, the next step can still look like this:

```
Entity A                Entity B
   |                       |
build form              build form
collect data            collect data
normalize                normalize
submit                  submit
   |                       |
Entity A                Entity B
   |                       |
validate                validate
authorize                authorize
CRUD                     CRUD
reply                   reply
```

The declarations are no longer duplicated.

The procedures consuming them still are.

R2WX pushes the shared structure one step further:



The entities still exist.

Their differences still exist.

What begins disappearing is the need for each one to independently implement behavior the infrastructure already knows how to perform.

On the frontend, the form became machinery

CRUD forms tend to contain a lot of work that is important but unsurprising.

Inputs need to be rendered.

Their constraints need to be respected.

Current values need to be represented.

The form needs to know whether it is adding, reading, editing or deleting.

Submission needs to collect the appropriate values and send them through the appropriate operation.

R2WX concentrates that recurring work into shared form machinery rather than treating every entity operation as a new form implementation.

Its operational states are explicit:

```
ADD
READ
EDIT
DEL
```

But these do not require four unrelated implementations.

They are states exercised through common machinery:

Implementation evidence — CRUD as states of shared form machinery

```
export const FORM = {
   ADD:  1,
   READ: 2,
   EDIT: 3,
   DEL:  4
};
```

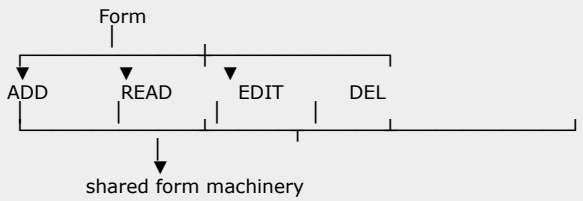
```
let disabled = $derived(
   [FORM.READ, FORM.DEL].includes($form.mode)
);

<fieldset disabled={disabled}>
   <R2wxInput {id} />
   ...
</fieldset>

{#if $form.mode === FORM.ADD}
   <button onclick={() => formPost({ uri: ctx.uri.post })}>
      <strong>submit</strong>
   </button>
{/if}

{#if $form.mode === FORM.EDIT}
   ...
{/if}

{#if $form.mode === FORM.DEL}
   ...
{/if}
```



That changes the development task.

Instead of repeatedly implementing:

“How does an edit form work?”

the recurring behavior already exists.

The entity provides the knowledge and the particular representation needed for this exercise.

CREATE, READ, UPDATE and DELETE became operational states of common form machinery rather than four independent form implementations.

That is what form-lifecycle means here.

Not merely that a form has states.

The states participate in reusable procedure.

The backend followed the same direction

The same pressure exists after the request crosses the boundary.

A conventional entity API repeatedly needs to answer familiar questions:

Is this endpoint exposed?

Is this caller authorized?

Which part of the request belongs to data and which to files?

Do the submitted values satisfy the active rules?

Which service operation should execute?

How should the result be returned?

Writing those answers again for every entity would recreate procedural repetition behind a now-shared application model.

R2WX instead places the recurring CRUD machinery in BaseController and the corresponding persistence operations in BaseService.

Implementation evidence — shared backend procedure

```
@Get(PATH.ID)
async getID(
    @Param('id', ParseIntPipe) id: number,
    @Req() req: RequestAuthz
) {
    this._assureEndpointAccessible(HTTP.GET, PATH.ID, req.userAccess);

    const data = await this.service.findUnique({
        where: { id }
    });

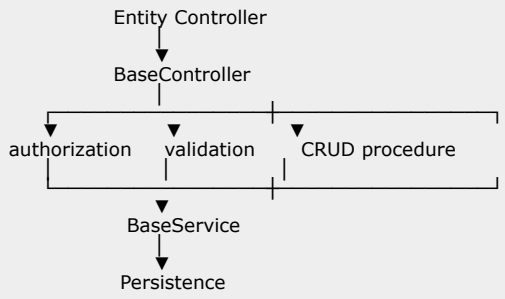
    return { data };
}
```

```
export class BaseService {
    constructor(protected db: DbService, protected entity: string) {
        this.model = db[entity];
    }

    async findUnique(clause: {}) {
        return this.model.findUnique(clause);
    }

    async update(...) { ... }
    async delete(...) { ... }
    ...
}
```

Conceptually:



An entity controller therefore does not need to rediscover the ordinary mechanics of CRUD simply because a new entity exists.

The shared infrastructure already carries them.

Validation stopped being rewritten around every operation

Validation exposes the same problem particularly clearly.

A complete operation may need the complete relevant scheme.

A partial operation may need only some of it.

One response would be to build another validator for the partial case.

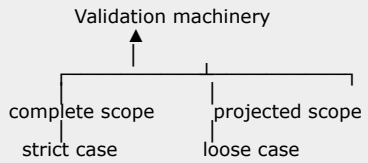
Then another representation.

Then another maintenance point.

R2WX takes a different route.

Its strict validation procedure remains reusable.

For a partial operation, the scope presented to that procedure changes.



The significant move is:

> **Change the scope. Reuse the procedure.**

Implementation evidence — project the scope, reuse the procedure

```
// Validation.ts

strict(data: {}, scheme: {}, optionalExcludes = []) {
    // validate data against the supplied scheme
    ...
}

loose(data: {}, scheme: {}, optionalExcludes = []) {
    const schemeLoose = {};
    const keys = Object.keys(data).filter(key => key !== 'id');

    keys.forEach(key => {
        if (key in scheme)
            schemeLoose[key] = scheme[key];
        ...
    });

    return this.strict(data, schemeLoose, optionalExcludes);
}
```

Rather than creating a second validation engine for partial data, R2WX projects the applicable fields and delegates back into the existing strict machinery.

The same pattern appears elsewhere in the artifact:

adapt the input, configuration or context around reusable procedure rather than reproducing the procedure itself.

That is more than code reuse.

It changes where new application work is expected to happen.

The developer stops implementing the ordinary path

At this point, a new entity no longer begins from an empty page.

R2WX already has expectations about:

- how common forms behave;
- how CRUD state moves through them;
- how values are prepared;
- how requests reach the backend;
- how recurring validation works;
- how controllers execute conventional entity operations;
- how services reach persistence.

So the development shape moves from:

```
NEW ENTITY
   |
   ▼
implement form behavior
implement submission
implement validation
implement controller procedure
implement CRUD
implement persistence interaction
```

toward:

```
NEW ENTITY
   |
   ▼
describe / attach entity
   |
   ▼
existing infrastructure
interprets the description
   |
   ▼
recurring procedure already available
```

There is an important boundary here.

R2WX does not generate an entire application from a scheme.

Entities still need to be instantiated and wired into the system.

Folders, routes, references and entity-specific configuration still exist.

The reduction is in implementation, not existence.

> **R2WX reduces repeated implementation. It does not eliminate application instantiation.**

Implementation evidence — implementation reduced, instantiation retained

```
@Controller(path)
export class SamxController extends BaseController {
    constructor(service: SamxService) {
        super(service, type, api);
    }
}
```

```
@Injectable()
export class SamxService extends BaseService {
   constructor(db: DbService) {
      super(db, entity);
   }
}
```

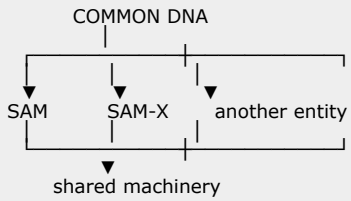
That distinction matters because otherwise schema-driven infrastructure can easily be mistaken for application generation.

That is not what this artifact demonstrates.

Different entities can now exercise the same machinery

Once recurring procedure moves downward into infrastructure, entity differences stop requiring entirely different execution paths.

Conceptually:



The visible entity can change.

Its fields can change.

Its representation can change.

Yet substantial portions of form handling, validation, controller behavior and persistence procedure remain the same.

This is where R2WX's crud-application character is useful—but also where it can be misleading if treated as the purpose of the artifact.

CRUD provides a highly repetitive environment.

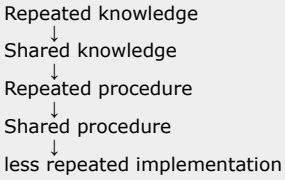
That makes it an effective place to ask:

Which parts of application work genuinely change between entities, and which parts are developers repeatedly implementing because the infrastructure has not absorbed them yet?

In R2WX, CRUD became the practical foundation on which that question could be exercised.

But automation creates its own battle

By this point, the direction looks attractive:



Keep following that direction and an obvious temptation appears:

If automating this much helped...

why not automate the rest?

That is where the next problem begins.

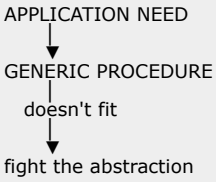
Because applications eventually do something that does not fit the ordinary path.

A particular form needs a different representation.

A submit operation needs to prepare data differently.

An entity requires behavior that the generic procedure did not anticipate.

And suddenly the infrastructure that removed repetitive work can begin creating a different kind of work:



That is a battle R2WX does not try to solve by making the abstraction increasingly absolute.

It deliberately leaves somewhere for the developer to intervene.

So the next question is no longer:

How much recurring work can infrastructure absorb?

It becomes:

How much should it absorb before the infrastructure itself starts getting in the developer's way?

WHEN AUTOMATION STARTED FIGHTING THE APPLICATION

extensible-architecture · extension-points · controlled-specialization · form-lifecycle

THE BATTLE

Automation feels successful while the application keeps asking for things the automation already understands.

Then one requirement doesn't fit.

A form needs a different structure. A submission needs data prepared differently. An entity needs additional procedure around the ordinary path.

At that point, highly automated infrastructure can create an uncomfortable choice:

duplicate the machinery outside the abstraction, or keep modifying the abstraction until one exceptional requirement becomes everybody's concern.

This was not a theoretical risk for the direction R2WX was taking.

Section 3 showed the attraction clearly:

```
describe recurring knowledge
        ↓
reuse recurring procedure
        ↓
write less application code
```

But that direction has a dangerous endpoint:

```
more automation
      ↓
more assumptions
      ↓
more cases forced through those assumptions
      ↓
the infrastructure starts deciding
what the application is allowed to be
```

I had already encountered the result of pushing automation too far.

The infrastructure that was supposed to remove repetitive work could become a personal prison: efficient while the application remained inside its assumptions, increasingly resistant when it did not.

That produced a deliberate rule for R2WX:

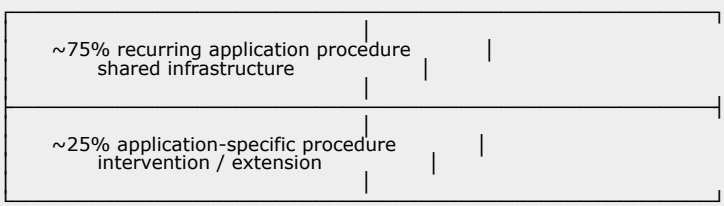
> **Never try to automate everything.**

The remaining space was not unfinished automation.

It was reserved intervention space.

The 75/25 direction

R2WX therefore works toward an approximate division:



The numbers are not measurements.

They are not a required ratio for every application.

They describe the engineering target:

Let infrastructure absorb the dominant recurring path while deliberately preserving room for application-specific procedure.

As an application deviates further from the recurring path, the shared proportion can decrease.

Nothing requires an application to manufacture itself into the 75%.

The important part of 75/25 is therefore not 75.

It is the refusal to eliminate the 25.

The first escape is representational

The frontend makes this visible through Generic and Custom forms.

Implementation evidence — generic representation and custom composition

```
<!-- R2wxFormGeneric.svelte -->

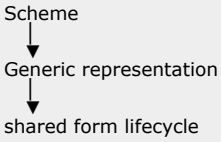
<form class="r2wx-form">
   {#each Object.values($active.form) as attr}
      <R2wxFieldsetInput id={attr.id}/>
   {/each}
</form>
```

```
<!-- R2wxFormCustom.svelte -->

<form class="r2wx-form">
   <slot/>
</form>
```

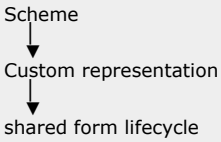
```
<R2wxFormCustom ctx={{ uri: ... }}>
   <R2wxFieldsetInput id="name"/>
   <R2wxFieldsetInput id="number"/>
   ...
</R2wxFormCustom>
```

When the ordinary representation is sufficient, generic machinery can construct the form from the application knowledge already available:



But representation itself can be application-specific.

R2WX therefore also allows:



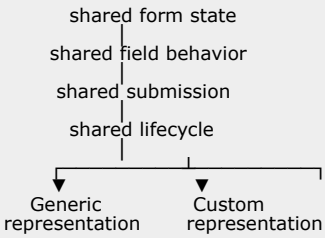
The distinction is important.

Generic does not mean 75%.

Custom does not mean 25%.

They are one concrete place where the broader design becomes visible.

Both can participate in the same surrounding machinery.



The application can therefore reject the generic representation without having to reject the entire infrastructure surrounding it.

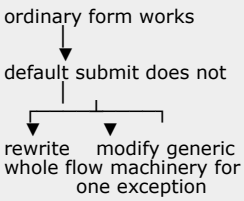
That is the first important property of the escape hatch:

> **Specializing one layer does not automatically require abandoning every shared layer around it.**

Sometimes the form is fine and the procedure is not

Another requirement may fit the ordinary form perfectly but need different preparation before submission.

Without an intervention point, the choices become poor:



R2WX exposes $action.submit as an intervention point.

The default path can resolve form data, normalize it and choose the appropriate request preparation.

But application code can replace the automatic payload derivation when necessary and still continue through the surrounding submission lifecycle.

Implementation evidence — replace payload derivation, retain the submission path

```
if ($action.submit)
    data = $action.submit();

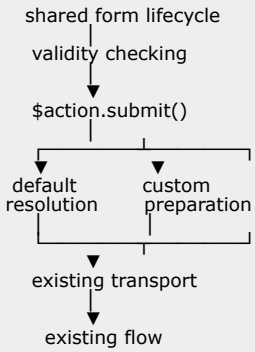
res = await formPost({ uri, data });
```

```
export const formPost = async (ctx) => {
    if (!canSubmit())
        return formReport();

    const data = ctx.data || _resolveFormData();

    return httpClient.post(ctx.uri, data);
};
```

Conceptually:



The developer does not have to choose between:

accept all automation

and:

abandon all automation.

The intervention can happen at the depth where the application actually differs.

That creates several depths of participation

Across materially different parts of R2WX, the same broad pattern appears.

Sometimes the shared procedure can simply be accepted.

Sometimes existing capabilities can be composed differently.

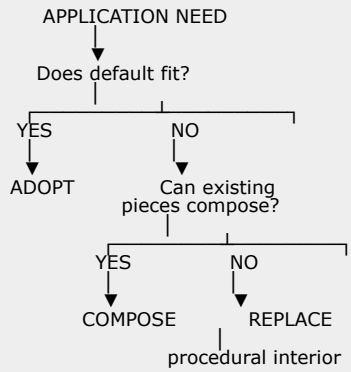
Sometimes part of the ordinary procedure can be replaced.

Implementation evidence — override one backend procedure

```
export class UserService extends BaseService {
   constructor(db: DbService, private svId: IdentifierService) {
      super(db, entity);
   }

   override async create(entry) {
      return this.model.create({
         data: {
            email: entry.email,
            password: await this.svId.argon2Hash(entry.pass)
         }
      });
   }

   // findFirst(...) ...
}
```



These are not three formal R2WX APIs bearing those names.

They are a useful way of describing the different depths of intervention visible across the implementation.

That distinction matters.

The graduated freedom exists in the artifact.

The three-level model is a way of understanding it.

But freedom stops at the receiving boundary

An escape hatch cannot mean:

custom code is exempt from the system.

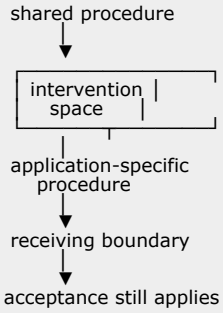
If application-specific procedure ultimately sends data to a backend boundary, that boundary still decides what it accepts.

If persistence receives data, its own constraints still apply.

So the complete picture is not:

```
AUTOMATION
   ↓
ESCAPE
   ↓
anything goes
```

It is:



This gives R2WX an important distinction:

> **An exception to procedure is not an exception to correctness.**

Implementation evidence — custom procedure still meets the receiving boundary

```
@Post("authenticate")
async authenticate(@Body() data, @Req() req: RequestAuthz) {
   this._assureEndpointAccessible(
      HTTP.POST,
      PATH._,
      req.userAccess
   );

   this._assureDataValid(data, VALID.STRICT);

   return this.service.findFirst(data);
}
```

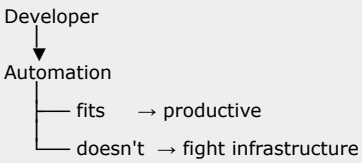
A custom interior may choose another route to the boundary.

It does not acquire the authority to redefine that boundary merely because it stopped using the default procedure.

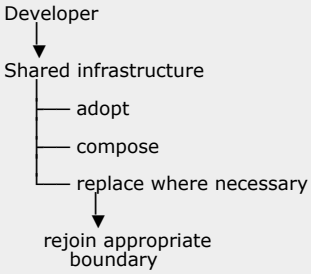
The developer keeps somewhere to stand

This changes the role of automation.

Without extension space:



With controlled intervention:



The infrastructure can continue removing repetitive implementation without claiming ownership over every possible application procedure.

This is where extensible-architecture and extension-points become useful descriptions.

But again, the concepts follow the development problem.

The original question was simpler:

How can recurring work disappear without making exceptional work harder than it was before?

R2WX's answer was not to weaken the shared machinery.

It was to preserve controlled places where the developer can stop following it.

The 25% is therefore not failure

This is perhaps the most important part of the 75/25 idea.

A conventional automation metric might interpret this:

75% automated

25% handwritten

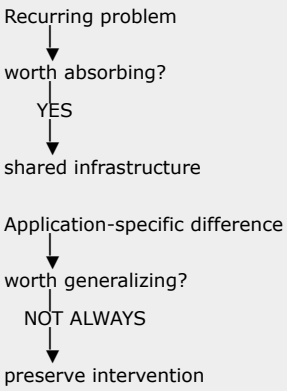
and ask:

Why isn't the remaining 25% automated yet?

R2WX asks a different question:

Does automating that remaining procedure still remove repetition, or has automation started removing useful freedom?

That makes the non-automated space deliberate.



The target is not maximum automation.

It is useful automation without captivity.

The amount of infrastructure R2WX can provide is not the amount the application is required to accept.

And then another problem appeared

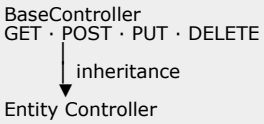
Preserving developer freedom solves the problem of over-automation.

But reusable infrastructure continues accumulating capability.

BaseController, for example, can know how to perform several operations.

Different entity controllers inherit that machinery.

Now a different question appears:



The application may want the implementation.

That does not necessarily mean its API should expose everything that arrived with it.

So after fighting one danger:

shared procedure becoming compulsory procedure

R2WX encounters another:

shared capability becoming exposed capability.

And that moves us directly into the next battle:

When Inheritance Started Giving Too Much

where the distinction becomes:

Inheritance supplies capability. Configuration grants exposure.

WHEN INHERITANCE STARTED GIVING TOO MUCH

default-deny · least-privilege · access-control · api-design · separation-of-concerns

THE BATTLE

Reusable base classes are useful precisely because they give extending classes capabilities they no longer have to implement themselves.

But inheritance is generous.

Implementation evidence — capability exists in the base

```
// BaseController.ts

export class BaseController {
    @Get()
    async get(@Req() req: RequestAuthz) {
        this._assureEndpointAccessible(
            HTTP.GET,
            PATH._,
            req.userAccess
        );

        const data = await this.service.findMany();
        return { data };
    }

    // @Get(PATH.ID) ...
    // @Post(PATH.MULTI) ...
    // @Put(PATH.ID) ...
    // @Delete(PATH.MULTI_ID) ...
}
```

As the base grows, every extending controller receives more machinery:

GET · POST · PUT · DELETE · ...

An entity may need only some of it.

Now the developer has another responsibility: remember which inherited capabilities should actually be available, and restrain the ones that should not.

This is a different problem from Section 4.

There, R2WX wanted the developer to remain free to leave the generic procedure when the application required something different.

Here, the concern runs in the opposite direction:

Reusable infrastructure can provide more capability than the application intends to expose.

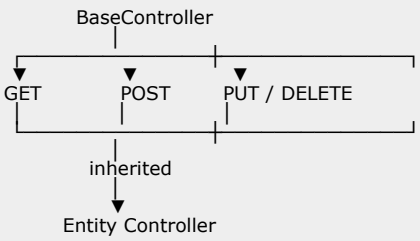
The machinery is useful.

Automatic exposure of everything the machinery knows how to do is not.

A base controller naturally accumulates capability

As recurring CRUD procedure moves into BaseController, its value increases because more controllers can inherit the same implementation.

Conceptually:



That solves the implementation problem.

The entity controller does not have to recreate those operations.

But there are now two different questions hiding inside what initially looks like one decision:

Can this controller perform the operation?

and:

Should this entity API expose the operation?

Inheritance answers the first.

R2WX refuses to let inheritance automatically answer the second.

Inherit the machinery. Expose nothing by inheritance alone.

The backend configuration introduced earlier now becomes more significant.

An entity can declare the API operations it intends to expose:

Implementation evidence — exposure is explicitly admitted

```
// stat_owner/config.ts

export const api = {
    GET: {
        [PATH._]: [ACCESS.GLOB]
    }

    // ...
};
```

```
// StatOwner.controller.ts

@Controller(path)
export class StatOwnerController extends BaseController {
    constructor(service: StatOwnerService) {
        super(service, type, api);
    }
}
```

```
api: {
    GET: {
        [PATH._]: [ACCESS.GLOB]
    }
}
```

There is no corresponding declaration here for POST, PUT or DELETE.

That absence matters.

It does not mean:

```
POST exists publicly
        |
developer remembered
to disable it
        |
        ▼
POST disabled
```

Instead:

```
BaseController can perform POST
              |
              ▼
Entity API never declares POST
              |
              ▼
           SEALED
```

Nothing needs to be written to switch that operation off.

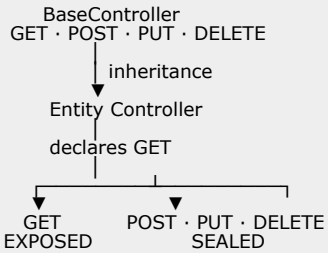
It was never switched on.

Capability and exposure became separate decisions

That gives R2WX a simple rule:

```
Inherited + declared     → EXPOSED
Inherited + not declared → SEALED
```

Or structurally:



The reusable implementation can therefore remain broad while the concrete API surface remains narrow.

```
Capability ≠ Exposure
```

This is the point at which default-deny becomes a precise description of the mechanism.

Possessing an implementation does not grant exposure.

A positive declaration does.

The rule is enforced, not merely documented

A configuration convention would be much weaker if every controller implementation still depended on the developer remembering to respect it.

R2WX places the check in shared controller procedure.

The request path goes through the assurance machinery:

Implementation evidence — exposure first, authorization second

```
// BaseController.ts

_assureEndpointAccessible(
    verb = HTTP.GET,
    path = PATH.ID,
    userAccess = ACCESS.GLOB
) {
    this._assureEndpointExposed(verb, path);
    this._assureUserAuthorized(verb, path, userAccess);
}

_assureEndpointExposed(verb = HTTP.GET, path = PATH.ID) {
    if (!(verb in this.api))
        throw ERR.VERB;

    const paths = Object.keys(this.api[verb]);

    if (!paths.includes(path))
        throw ERR.PATH;
}

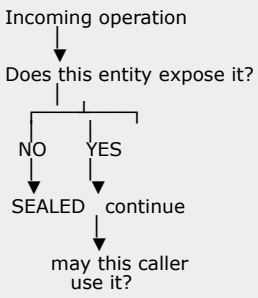
_assureUserAuthorized(...) {
    ...
}
```

```
this._assureEndpointAccessible(verb, path, req.userAccess);
```

with endpoint exposure verified through:

```
this._assureEndpointExposed(verb, path);
```

Conceptually:



This also separates another pair of questions that can easily become entangled:

Does this API provide the operation at all?

from:

Who is allowed to use it?

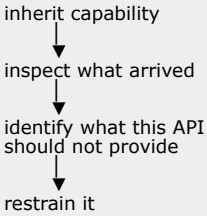
Exposure and authorization are related security concerns.

They are not the same decision.

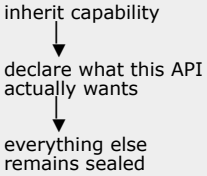
The developer now declares what is wanted, not what must be removed

Compare the maintenance burden.

A permissive inheritance model can leave the developer thinking:



R2WX reverses the direction:



This is why least-privilege is relevant here, but only in a specific sense.

R2WX is not claiming that inheritance itself disappears or that unused implementation is physically removed from the controller hierarchy.

The underlying reusable machinery can still exist.

The restriction concerns the API surface:

> **Reusable implementation can be inherited broadly while exposure is granted narrowly.**

That is the boundary of the claim.

Implementation ownership and exposure ownership separate

This also returns us to the ownership problem from Section 2.

`BaseController` owns reusable procedure:

HOW does GET work?

HOW does POST work?

HOW does PUT work?

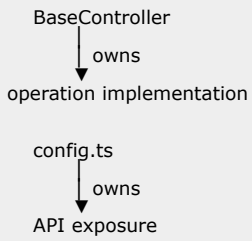
HOW does DELETE work?

The entity/application configuration owns a different question:

WHICH of these operations

belong to this API?

Those responsibilities do not have to live together.



So another R2WX distinction emerges:

```
Implementation ownership ≠ Exposure ownership
```

This is another concrete case of separation-of-concerns.

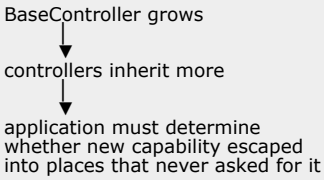
Sharing the implementation does not require sharing—or surrendering—the decision to expose it.

Growth no longer automatically expands every API

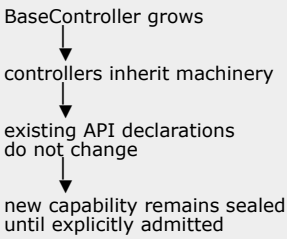
This becomes more important as shared infrastructure evolves.

Suppose BaseController gains another reusable capability.

Without an explicit admission boundary:



With R2WX's exposure rule:



That changes the direction of responsibility.

The developer does not have to continuously defend the API against everything reusable infrastructure learns how to do.

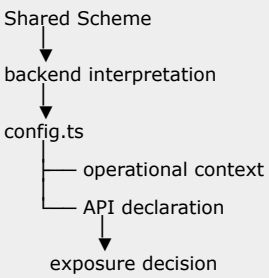
The API describes what it accepts into its surface.

This is where configuration becomes authority

Earlier, config.ts appeared simply as backend-owned interpretation around shared application knowledge.
Now one part of it acquires a stronger meaning:

```
api: {
   ...
}
```

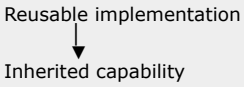
This is not merely convenient metadata about the controller.
It participates in deciding what inherited capability becomes externally available.
The progression we've followed now becomes visible:



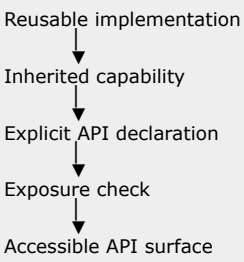
So configuration-driven in R2WX is not simply “behavior happens to live in configuration files.”
Configuration can own decisions that reusable procedure must obey.

The developer gets the machinery without inheriting the exposure burden

We started with a familiar attraction:
Put recurring CRUD operations into a base controller so they do not have to be implemented repeatedly.
That gives:



R2WX adds another boundary before that capability becomes part of an API:



And that produces the developer-facing consequence:

Get the reusable machinery. Carry only the responsibility of declaring what should be exposed.

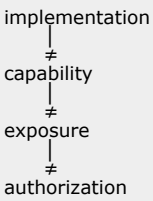
The infrastructure can grow.

The entity can inherit.

Neither event alone expands the public API.

But exposure is only one boundary

At this point R2WX has separated:



The next battle goes deeper than deciding whether an endpoint exists or which role may call it.

A browser user is interacting with the frontend.

The frontend is crossing into another service.

The backend needs to know whether that caller is trusted.

It would be easy to treat all of those as one identity moving unchanged through the system.

R2WX does not.

The human user's authority terminates at one boundary.

A different identity crosses the next.

So the next question becomes:

When a request crosses from a human-facing application into a protected core API, whose identity should actually cross that boundary?

That takes us into the BFF and R2WX's separation of human authorization from service authorization.

WHEN ONE USER REQUEST BECAME TWO DIFFERENT IDENTITIES

backend-for-frontend · security-boundaries · custom-authentication · stateless-authentication · access-control · separation-of-concerns

THE BATTLE

Authentication often begins simply.

A user logs in. The application knows who that user is. The user makes a request. The request eventually reaches the backend.

Then the system acquires another boundary:

```
Browser → Frontend/BFF → Core API
```

Now a deceptively simple question appears:

Whose identity is actually crossing each boundary?

The human is authenticated to the application.

But the service calling the protected backend is the BFF.

Treat those as the same identity and two different trust relationships begin collapsing into one.

R2WX keeps them separate.

The browser-facing side establishes and carries the human relationship.

The core API receives a request from a trusted service.

That creates two authorization problems rather than one authorization mechanism stretched across the whole topology.

The human authenticates to the application

The login begins with ordinary user credentials.

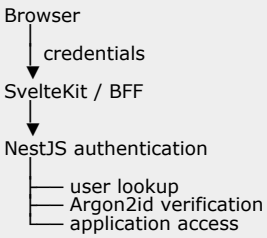
Those credentials ultimately reach backend authentication logic, where the stored user is retrieved and the password is verified using Argon2id.

Implementation evidence — human credentials are verified

```
override async findFirst(...) {
    const user = await this.model.findFirst(...);
    if (!user) throw 'NOUSER';

    const isAuthentic = await this.svId.argon2HashCompare(...);
    if (!isAuthentic) return null;

    delete user.password;
    return user;
}
```



Successful authentication gives the BFF what it needs to establish the browser-facing relationship.

R2WX then uses its own stateless signed ticket rather than introducing a server-side session store for the examined flow.

Implementation evidence — ticket construction topology

```
create(...) {
   const signature = this.makeDeviceSignature(...);

   return this.makeTicket(..., signature);
}

makeTicketContent(x) {
   const verificationHash = this.makeVerificationHash(...);

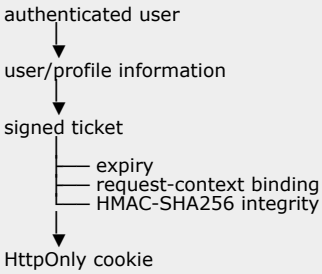
   return [x, ..., verificationHash];
}

makeTicket(...) {
   ...
}

makeDeviceSignature(...) {
   ...
}
```

The operational ingredients are intentionally abstracted here. The structure of the mechanism is the evidence required for this discussion.

Conceptually:



The ticket is not a JWT.

Its purpose here is narrower: carry the information required by the frontend-side authentication flow while protecting the ticket against undetected modification.

And because it is signed rather than encrypted:

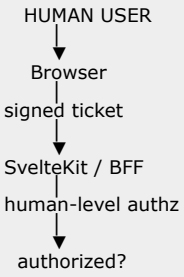
> **Integrity is provided. Confidentiality of the ticket contents is not.**

That distinction matters whenever custom authentication machinery is being inspected.

The BFF becomes the browser's security boundary

Once authenticated, the browser does not need to carry backend service credentials.

Instead:



The BFF can reason about the user-facing application profile and decide whether the human request is allowed to proceed.

Implementation evidence — human authorization remains at the BFF

```
accessRole(...) {
    this.access(...);
    if (!this.profile) throw 'ACC|NOPROFILE';

    const roleCurr = ...;
    const hasRoleAccess = ...;

    if (!hasRoleAccess) throw 'ACC|NOROLEACC';

    return this.profile;
}
```

That is one trust relationship:

```
Human identity → BFF authorization
```

But the request has not yet reached the core API.

Crossing that next boundary requires something different.

The core API does not receive the human as its service identity

A tempting design would be to keep propagating the human identity downward:

```
Human
  ↓
Browser
  ↓
BFF
  ↓
Core API
  |
"same identity everywhere"
```

R2WX instead changes identity at the service boundary.

The BFF calls the protected NestJS API using the credentials and authorization context assigned to the BFF/core relationship.

Implementation evidence — service identity crosses the next boundary

```
const headers = {
   'Authorization': SERVICE_CREDENTIAL,
   'X-User-Role': SERVICE_ROLE,
   ...
};
```

```
HUMAN
  ↓
Browser
  ↓
BFF
  |  human authorization ends here
  |
  |──────────────────────────────────
  |     SERVICE BOUNDARY
  |──────────────────────────────────
  |
  |  trusted service identity
  ↓
NestJS Core API
```

So the second relationship is:

```
BFF service identity → Core API authorization
```

Implementation evidence — the core establishes backend access from the service relationship

```
canActivate(context: ExecutionContext): boolean {
  const req = context.switchToHttp().getRequest();

  // ... service credential verification ...

  const userRole = ...;
  if (!userRole) throw new ForbiddenException('NOROLE');

  const accessLevel = ...;
  if (!accessLevel) throw new ForbiddenException('NOACC');

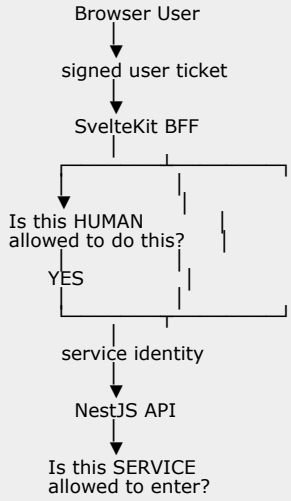
  req.userAccess = accessLevel;

  return true;
}
```

The backend sees the trusted service relationship rather than treating the browser user's profile as the service credential crossing into the core.

One request, two questions

This makes the complete path easier to understand:



These questions can occur during what the user experiences as one operation.

Architecturally, however, they belong to different boundaries.

That is where backend-for-frontend becomes more than a routing label in R2WX.

The BFF absorbs a trust transition.

Why not simply forward the human role?

Because the core API and the BFF do not have the same relationship with the human.

The BFF is the human-facing application boundary.

The core API is protected behind it.

Propagating the browser user's application role as though it were the credential establishing trust with the core would collapse:

```
Who is the human?
      |
      ≠
May the human perform this UI/application operation?
      |
      ≠
Which service is calling the core?
      |
      ≠
May that service enter this backend boundary?
```

R2WX keeps those questions structurally distinct.

> **Human authorization and service authorization are different responsibilities.**

That is another instance of a pattern we've already encountered.

Section 2 separated shared application knowledge from runtime-owned interpretation.

Section 5 separated inherited capability from API exposure.

Here:

User identity is separated from service identity.

Again, sharing a request path does not require collapsing the responsibilities encountered along that path.

The custom ticket solves a specific problem

Because R2WX owns the browser-facing authentication flow, it also owns the ticket mechanics.

The ticket carries an expiry and is associated with request context.

The context signature incorporates information such as the request's User-Agent and Accept-Language, together with server-side secret material.

Conceptually:

```
ticket
  ├── user/profile information
  ├── expiry
  ├── context association
  └── HMAC signature
```

On subsequent requests:

```
incoming ticket
      ↓
signature valid?
      ↓
expired?
      ↓
expected context?
      ↓
human-facing request accepted
```

The context association should not be mistaken for strong device identity.

User-Agent and language headers do not establish that.

What it does demonstrate is that R2WX's ticket acceptance considers more than possession of an unexpired value.

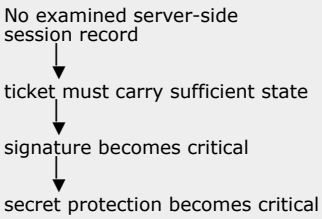
The ticket can also be renewed as requests continue through the flow.

This is the concrete basis for describing the examined mechanism as custom-authentication and stateless-authentication.

Stateless does not mean trustless

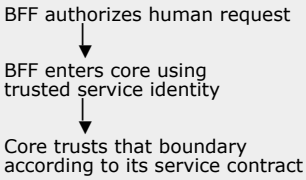
Removing server-side session state doesn't remove authority from the system.

It relocates what must be trusted.



Likewise, separating service identity from human identity doesn't make the core independently aware of everything the BFF already decided.

It creates a trust relationship:



That has a consequence worth making explicit:

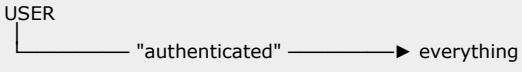
Compromise of the BFF would compromise the trust relationship through which it reaches the protected core.

The boundary provides separation.

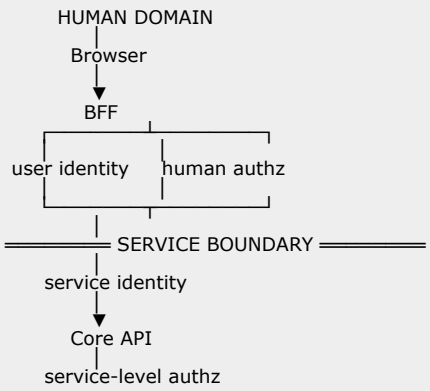
It does not make the participating service magically harmless.

The developer can now reason about one boundary at a time

Without this separation, authentication can become a single conceptual blob:



R2WX instead produces a topology:



Now the questions become local:

At the browser/BFF boundary: who is the human, and what may that human do?

At the BFF/core boundary: who is the calling service, and may that service enter?

That is what security-boundaries means in this part of R2WX.

The architecture follows the trust relationships rather than assuming one authentication result should remain authoritative everywhere.

Authority follows the boundary in which that authority has meaning.

And then the payload becomes the problem

Once the BFF owns this boundary, another practical responsibility appears.

Requests have to pass through it.

For ordinary JSON, that may seem trivial.

Then a form contains files.

Payloads become multipart.

A file can become large.

Dynamic form data may need server-loaded options.

And suddenly a BFF that was introduced partly to protect and mediate the backend can itself become unnecessary work:



The next battle therefore moves from who may cross the boundary to:

How can the boundary remain in place without forcing every payload to pay for the boundary unnecessarily?

That takes us into R2WX's runtime and transport adaptation: stream forwarding, automatic JSON/multipart selection, scheme-derived file limits and runtime-loaded options.

WHEN THE BOUNDARY STARTED DOING TOO MUCH WORK

http-streaming · multipart · schema-driven · dynamic-configuration · extension-points

THE BATTLE

Put a BFF between the browser and the backend and every request now has somewhere else to pass through.

For a small JSON payload, that hardly attracts attention.

Then the form contains files.

The browser sends multipart data. The BFF can parse it, materialize the files and fields, reconstruct another request, and send everything to the backend.

But if the BFF does not actually need to understand that payload, all of that work exists only because the request happened to cross the BFF.

This is a familiar danger when infrastructure gains responsibility.

A boundary introduced for one reason can quietly start accumulating work unrelated to that reason.

In R2WX, the BFF has important responsibilities around the browser-facing trust boundary.

That does not mean it should automatically become the owner of every payload passing through it.

So the transport path needed to adapt without dismantling the boundary itself.

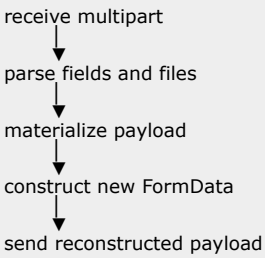
Keep the boundary. Stop unnecessarily materializing the payload.

For multipart forwarding, R2WX uses the incoming request body directly when proxying toward NestJS:

```
fetch(url, {
    method,
    body: request.body,
    duplex: "half"
});
```

The significant part is what does not happen first.

The BFF does not need to:



The path can instead remain:

```
Browser
   |
   | multipart body
   ▼
BFF
   |
   | stream passthrough
   ▼
NestJS
   |
   ▼
multipart handling
```

The BFF remains in the request path.

Its security and routing responsibilities remain.

But it does not take ownership of multipart interpretation merely because the bytes cross its boundary.

Implementation evidence — forward the body without parsing it

```
return fetch(this.url(path, ENV_DATASERVER), {
    headers: ...,
    body: request.body,
    method,
    duplex: 'half'
});
```

> **Crossing a component does not require becoming that component's responsibility.**

This is the concrete R2WX basis for http-streaming here.

The claim should remain precise: the unparsed request body is forwarded as a stream, avoiding multipart parsing and FormData reconstruction in the frontend container.

It does not establish that no buffering can occur anywhere in the complete network/runtime stack.

The form should not need to decide transport manually either

Implementation evidence — default form machinery chooses representation

```
const _resolveFormData = () => {
    return document.querySelector('input[type=file]')
        ? makeMultipartForm()
        : collectFormData();
};
```

The same problem appears one step earlier.

A conventional form may contain only scalar values:

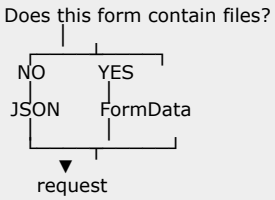
```
name
email
active
```

Another may contain:

```
name
description
file
```

The presence of a file changes how the request must be represented.

Without shared handling, application code begins accumulating transport decisions:



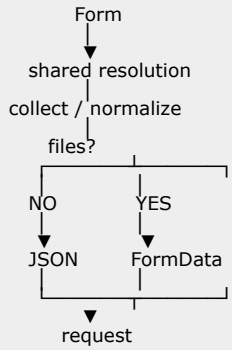
There is nothing intellectually difficult about writing this.

That is exactly why it is a good candidate for infrastructure.

It is recurring work whose answer can be derived from the form being submitted.

R2WX's default form resolution collects the values, normalizes them, identifies file inputs and prepares the appropriate payload.

Conceptually:



The application doesn't need to repeatedly implement the ordinary transport decision.

This is automatic JSON/multipart selection.

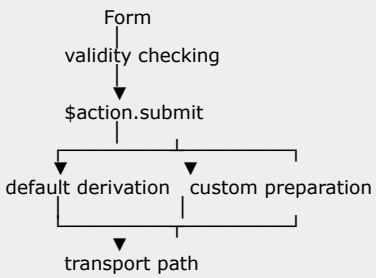
It is not HTTP content negotiation.

The client-side machinery is selecting an appropriate request representation from the actual form payload.

And the escape hatch from Section 4 still survives

Automation is useful until this particular submission does not fit the default derivation.

That is why the earlier $action.submit intervention point matters again.



We no longer need to explain the entire extension mechanism.

Section 4 already did that work.

Here it demonstrates progressive loading:

Transport automation is available without making automatic payload derivation compulsory.

The infrastructure can solve the ordinary transport problem while leaving the application somewhere to intervene when the payload has different requirements.

File handling pushes application knowledge into transport infrastructure

Once the request reaches NestJS, multipart handling has another practical requirement.

File size limits have to be established before ordinary application validation can simply inspect an accepted payload.

One option would be another independent configuration:

Scheme says:

file max = X

Multer config says:

file max = Y

Now we have returned to the battle from Section 1.

The same application rule has acquired another independently maintained representation.

R2WX instead derives the applicable Multer fileSize limit at runtime from the active entity scheme's file-size rules.

Implementation evidence — scheme rules reach multipart configuration

```
const { ruleFile } = this._splitRules();

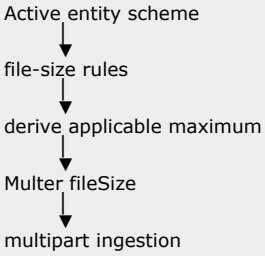
await this.svMulter.processHttp(
    req,
    res,
    ruleFile
);
```

```
const fields = Object.values(rules);

const maxSchemaSize =
   Math.max(...fields.map(field => field.size));

const upload = multer({
   dest: ...,
   limits: { fileSize: maxSchemaSize }
});

...
```



The same knowledge describing what files the application accepts now participates in configuring the infrastructure that receives them.

Application knowledge reaches the transport boundary before the payload does.

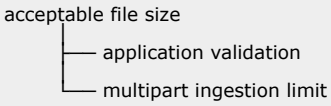
That is the interesting part of the mechanism.

It is not dynamically resizing physical infrastructure.

It is not changing memory allocation.

It is schema-derived multipart configuration.

And once again the developer is relieved of another synchronization point:



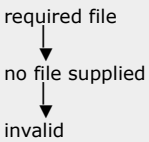
Both can originate from the same application knowledge.

Requiredness changes meaning when a file already exists

Files expose another battle that ordinary scalar fields can hide.

Suppose an entity requires a file.

During creation:



Straightforward.

Now edit an existing entity.

The entity already has its required file.

Should opening the edit form force the developer or user to upload that file again merely because the field is structurally required?

No.

The entity requirement and the interaction requirement are no longer identical.

```
ENTITY REQUIREMENT

entity must have a file
        |
        ≠
        |
INTERACTION REQUIREMENT

every edit must supply
a replacement file
```

R2WX's file-input handling recognizes this distinction in its update behavior.

Implementation evidence — runtime interaction modifies requiredness

```
{@const { value, ...inputCurr } = input}

<input
   {...inputCurr}
   required={input.value ? false : inputCurr.required}
   ...
/>
```

That gives us another useful finding:

```
Entity requiredness ≠ interaction requiredness
```

The entity can require the resource to exist while an update interaction requires a new file only when replacement is actually intended.

Again, one word—required—does not automatically mean the same thing at every layer.

Context determines which responsibility is being exercised.

Some schemes need information that does not exist until runtime

Not every valid form option can be known statically.

A select field may depend on records currently stored in the database.

Hardcoding those values into the scheme would make the declaration stale.

Requiring every form to independently fetch and assemble them would reintroduce recurring procedure.

R2WX allows backend configuration to specify runtime option loading through _option.

The backend can use the appropriate service to retrieve the current records and enrich the working scheme representation.

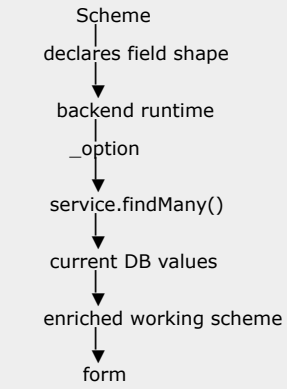
Implementation evidence — runtime data enriches the working scheme

```
async _loadSchemeOptions(dict) {
    const deps = await Promise.all(
        Object.values(dict).map(
            sv => sv.findMany({})
        )
    );

    const schemeNext = {
        ...this.scheme,
        ...Object.entries(dict).reduce((acc, [key, sv], i) => {
            acc[key] = {
                ...this.scheme[key],
                ['data-options']: deps[i].map(({ id }) => id)
            };

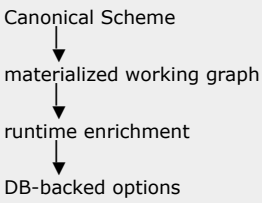
            return acc;
        }, {})
    };

    return schemeNext;
}
```



This is where the independently materialized scheme graphs from Section 1 become important again.

The runtime can enrich the working representation without mutating the canonical scheme definition or another consumer's materialized representation.



We introduced the clone mechanism earlier.

Now we see one reason that independence becomes useful.

Static definition and runtime reality can cooperate

Dynamic options reveal a broader distinction.

Some application knowledge is stable enough to declare:

```
field type
requiredness
bounds
option source
```

But the actual values may belong to current runtime state:

```
customer records
projects
categories
other DB-backed values
```

Trying to force both into static configuration would confuse two different kinds of knowledge.

R2WX instead allows:

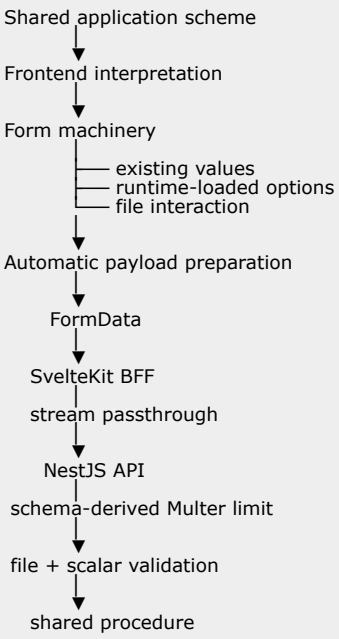
```
STATIC KNOWLEDGE
"What kind of field is this?"
"What source does it need?"
        ↓
RUNTIME PROCEDURE
"Load the current values."
        ↓
WORKING REPRESENTATION
```

That makes dynamic-configuration useful here in a limited sense.

The canonical description can remain stable while a runtime-specific working representation acquires information that only the running application can know.

Several earlier decisions now meet in one request

A multipart update can therefore cross much of what we've already established:



This is why transport deserves its own exploration topic.

It isn't an isolated file-upload feature.

It is where several earlier decisions are forced to cooperate under actual runtime conditions.

Shared knowledge.

Runtime-specific interpretation.

Reusable procedure.

Extension points.

Trust boundaries.

Transport.

They all meet when real data starts moving.

The boundary remains strict while the interior adapts

The developer-facing pattern now looks familiar.

R2WX does not require every application operation to manually decide:

JSON or multipart?

How do I collect it?

How do I proxy it?

How large may the file be?

How do I populate dynamic options?

Does required mean replacement?

Recurring answers move into infrastructure.

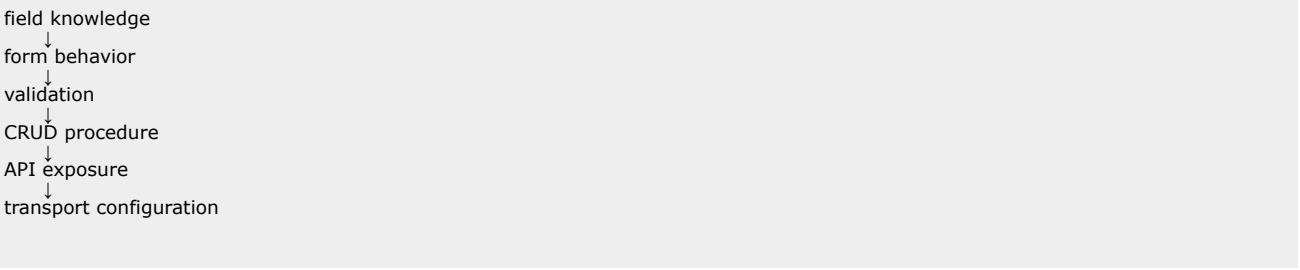
But the application still has intervention points when the default procedure does not fit.
So the transport layer reinforces something Section 4 already exposed:

> **The interior can adapt. The receiving boundary still decides what it accepts.**

The path can change.
The representation can change.
Runtime knowledge can enrich the working structure.
A custom submit can replace automatic preparation.
But when the request reaches the backend, backend acceptance still applies.

And eventually consolidation reaches a boundary it deliberately does not absorb

Across the article so far, R2WX has repeatedly moved recurring knowledge or procedure into shared infrastructure:



At this point, it would be natural to ask:

If the application already knows so much about its entities, why not let that knowledge become the final authority all the way down?

Why maintain another model in Prisma?

Why tolerate another alignment point between application schemes and persistence?

Why not generate one directly from the other?

R2WX deliberately stops.

And that final refusal is important because it reveals that removing duplication was never an absolute rule.

The last exploration topic is therefore:

When Consolidation Stopped at Persistence

R2WX finally encounters a repeated representation it chooses to keep.

Because at that boundary, independence is no longer treated merely as duplication.

It becomes protection.

WHEN CONSOLIDATION STOPPED AT PERSISTENCE

separation-of-concerns · schema-validation · prisma · data-boundaries · persistence

THE BATTLE

By now, R2WX has spent much of the application path eliminating independently maintained versions of knowledge it already possesses.

A field constraint should not need to be separately rediscovered by the form, the frontend, the backend and the transport layer.

So eventually an obvious inconsistency appears.

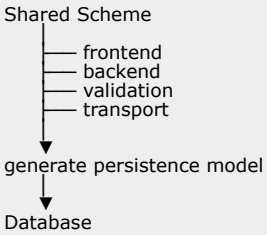
The database model still has its own definition.

Prisma still describes what can be persisted.

That means changing an entity can once again require another deliberate change somewhere else.

After spending so much effort reducing exactly that kind of synchronization work, why stop here?

The tempting continuation is obvious:



One description.

One propagation path.

No final synchronization point.

R2WX does not take that step.

And that refusal exposes an important limit to everything we have explored so far.

Repeated knowledge was never the enemy by itself.

The question was whether independently maintaining it served a responsibility worth preserving.

At persistence, R2WX answers yes.

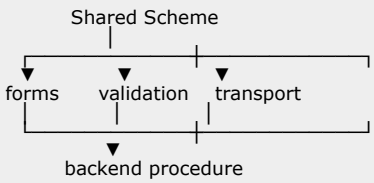
The application knows a great deal about an entity

By the time data reaches persistence, the application has already exercised substantial knowledge about it.

The shared schemes can describe things such as:

```
type
requiredness
accepted values
bounds
file rules
relationships used by application behavior
```

That knowledge has already participated in multiple procedures:



It would therefore be technically conceivable to keep pushing that authority downward.

But persistence represents a different question.

The application asks:

Is this acceptable application data for this operation?

Persistence ultimately asks:

Can this state actually exist here?

Those questions overlap.

They are not identical.

Prisma keeps its own model

R2WX retains schema.prisma as an independently maintained persistence definition.

Implementation evidence — the same entity remains independently represented

Application scheme:

```
// schemes/samx/index.js

name: {
    ...T.TEXT,
    maxLength: 160
},

status: {
    ...T.SELE,
    required: true,
    'data-options': [
        'DRAFT',
        'READY',
        'ACTIVE',
        'PAUSED',
        'CLOSED'
    ],
    value: 'DRAFT'
},

...
```

Persistence model:

```
// schema.prisma

model TOOL_SAMX {
   id   Int    @id @default(autoincrement())
   name String @db.VarChar(160)

   status ENUM_STATUS @default(DRAFT)

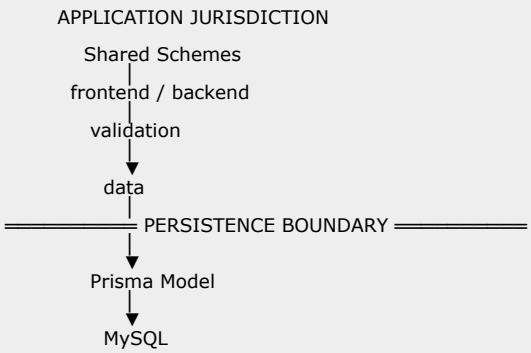
   // ... additional fields ...

   category_id  Int
   StatCategory STAT_CATEGORY
      @relation(fields: [category_id], references: [id])

   owner_id Int?
   Owner OWNER?
      @relation(fields: [owner_id], references: [id])

   // ... timestamps ...
}
```

Conceptually:



The Prisma model is not generated from the application schemes.

Nor are the application schemes merely views generated from Prisma.

They have to remain aligned where their responsibilities overlap.

That creates real maintenance cost.

R2WX accepts it.

This is duplication with a purpose

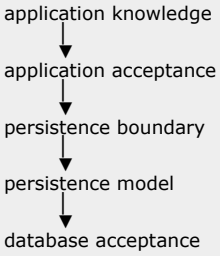
That distinction changes the meaning of the synchronization point.

Earlier, repeated declarations looked like this:

```
same application fact
   ├── copy A
   ├── copy B
   ├── copy C
   └── copy D
```

Each copy created another opportunity for accidental divergence without necessarily adding another meaningful authority.

At persistence, the topology is different:



The second representation exists because another subsystem retains authority over what enters it.
So R2WX does not apply the earlier lesson mechanically:

```
duplication is bad → therefore eliminate every duplicate representation.
```

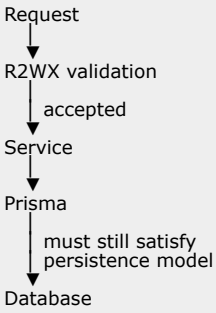
Instead:

> **Consolidate repeated knowledge where independent ownership serves no useful boundary. Preserve independent representation where independent authority is intentional.**

This is where the earlier theme of separation-of-concerns reaches its deepest boundary in the artifact.

Application validation does not get the final word

The backend's shared validation machinery can reject invalid application input before persistence.
That is useful.
It can provide application-specific semantics and errors.
It can understand rules expressed through the R2WX scheme vocabulary.
But passing that layer does not make the data inherently persistable.



So:

```
Application acceptance ≠ persistence acceptance
```

This also explains why R2WX services do not need to reproduce the complete application validation process yet again immediately before every persistence operation.

The layers are not three identical validators stacked together.

They perform different jobs.

Implementation evidence — application procedure hands off to Prisma

```
// BaseService.ts

export class BaseService {
   constructor(protected db: DbService, protected entity: string) {
      this.model = db[entity];
   }

   async create(entry = {}) {
      return this.model.create({ data: entry });
   }

   async update(...) { ... }
   async delete(...) { ... }
}
```

```
Frontend
   └── interaction / early conformance
Backend
   └── authoritative application acceptance
Prisma / Database
   └── persistence acceptance
```

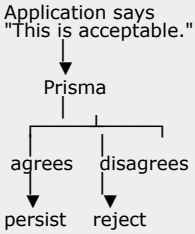
Authority changes with jurisdiction.

The final boundary can reject what the application thought was valid

This independence has a practical consequence.

If application knowledge and the persistence model drift apart, the discrepancy does not silently become database truth merely because the application accepted it.

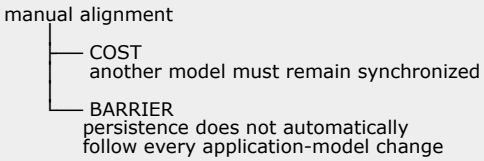
The persistence layer can resist.



That friction can be inconvenient during development.

But removing it would also remove an independent opportunity to catch disagreement before it becomes persisted state.

The maintenance point therefore performs two roles:



R2WX chooses to keep both.

Not every synchronization cost should be automated away

This is where the first exploration topic returns in a different form.

Section 1 began with:

Why should a developer repeatedly declare the same application fact?

The answer led toward consolidation.

Now we encounter:

Why should a developer have to align the application model with Prisma?

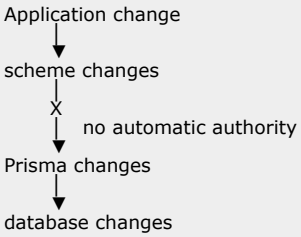
And R2WX gives a different answer:

Because this alignment crosses into a responsibility that was deliberately kept independently authoritative.

The manual step introduces resistance.

A persistence change cannot happen merely because an application scheme changed.

Someone has to cross that boundary deliberately.



That X is not simply missing automation.

In R2WX it is part of the boundary.

Persistence resistance is deliberate.

This also limits schema sovereignty

Throughout the earlier sections, the shared schemes have acquired considerable influence.

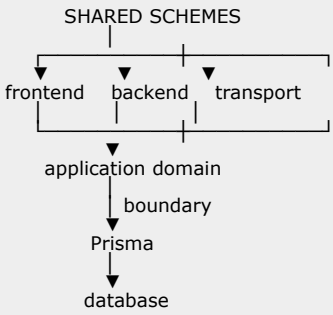
They participate in:

```
representation
validation
form behavior
runtime enrichment
file constraints
transport configuration
```

It would be easy by now to describe them as the single source of truth for the entire system.

That would be inaccurate.

Their authority has a jurisdiction.



So the stronger and more precise finding is:

> **The schemes are authoritative within the application responsibilities assigned to them. They are not sovereign over persistence.**

That qualification is not a weakness in the scheme model.

It is part of the design.

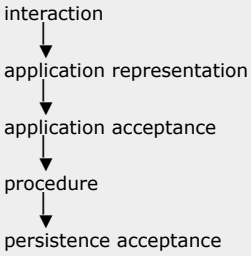
The database is not merely the bottom of the pipeline

If we read the architecture only as data flow, persistence looks like the final technical destination:

```
Browser → BFF → API → Service → Prisma → MySQL
```

But after following the eight explorations, another reading becomes possible.

The arrows are crossing responsibilities.



The database edge is therefore not interesting merely because it happens last.

It is interesting because R2WX deliberately stops propagating application authority before reaching it.

The final persistence edge remains independently sovereign.

And that resolves an apparent contradiction

At the beginning of this exploration, R2WX appeared to contain two opposing tendencies.

On one side:

remove duplication

centralize recurring knowledge

reuse procedure

automate repetitive work

On the other:

separate frontend/backend interpretation

preserve custom procedure

require explicit API exposure

separate human/service authority

retain independent persistence model

Those are not actually opposite strategies.

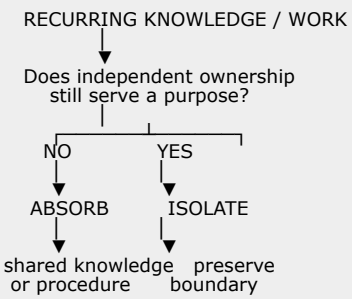
The dividing question is:

Does this repetition represent unnecessary repeated ownership, or does the separation preserve a responsibility that should remain independently owned?

When R2WX judges the former, it tends to consolidate.

When it judges the latter, it tends to preserve the boundary.

That gives the entire exploration a more precise shape:



Not everything repeated belongs together.

Not everything separated should remain separate.

The decision depends on responsibility.

Conclusion — What the Eight Pressure Points Leave Behind

R2WX began from familiar application work.

Fields need definitions. Forms need to represent them. Requests need validation. Entities need operations. Some operations need specialization. Endpoints need controlled exposure. Users need authorization. Payloads need to cross boundaries. Accepted data eventually needs to persist.

None of those concerns is unusual by itself.

The investigation became more interesting when they were examined together.

Across the eight pressure points, R2WX repeatedly encounters the same underlying decision from different directions: when should something that recurs become shared system capability, and when does separation still preserve a responsibility that matters?

Repeated application knowledge moves toward shared definitions, but the consumers of that knowledge retain decisions that belong specifically to them.

Repeated procedure moves into common infrastructure, but application-specific behavior retains explicit ways to depart from the generic path.

Reusable implementation provides capability, but configuration independently determines which of those capabilities an application exposes.

A human request can initiate an operation without requiring human identity to remain the authoritative identity across every boundary that operation crosses.

An intermediate layer can transport a payload without becoming responsible for interpreting its application meaning.

And after substantial consolidation elsewhere in the system, persistence remains independently represented because the final persistence boundary retains an authority of its own.

Considered together, these are not eight instances of the same rule being mechanically applied.

Sometimes R2WX absorbs a repeated concern into shared infrastructure.

Sometimes it isolates a responsibility that should remain independently owned.

The distinction does not follow from repetition alone.

It follows from responsibility.

From repeated implementation to setup

The practical consequence becomes visible in ordinary entity work.

When an entity fits the shared path, much of its standard lifecycle does not need to be implemented again. The entity still has to exist. Its schemes, configuration, routes and wiring still have to be established. Application-specific decisions still have to be made.

But setup and implementation are not the same thing.

Once recurring knowledge and procedure have become infrastructure, establishing another entity can increasingly mean describing and connecting it to machinery that already knows how to perform the recurring work, rather than reproducing that work around the new entity.

This is the territory represented by R2WX's approximate 75/25 design heuristic.

The ratio is not a measurement and is not invariant across applications. It expresses the intended balance: a substantial common path handled by reusable infrastructure, accompanied by deliberate space for behavior that remains specific to the application.

That remaining space is not treated as failure of the abstraction.

When specialized behavior is required, it can leave the generic procedure, perform the work that makes it exceptional, and—where the next contract allows it—rejoin the same shared lifecycle before reaching the receiving boundary.

The objective that survived the investigation is therefore not complete automation.

It is less repeated implementation without eliminating necessary engineering freedom.

What R2WX ultimately preserves

That distinction changes the meaning of several mechanisms that can otherwise look unrelated.

The shared schemes reduce repeated knowledge, but they do not erase frontend and backend responsibility.

The base machinery reduces repeated procedure, but it does not eliminate specialization.

Inheritance increases reuse, but it does not determine exposure.

The BFF participates in a request without transferring every form of authority through itself unchanged.

Streaming carries data without requiring every intermediate layer to own its interpretation.

And consolidation reaches persistence without claiming authority over the persistence model itself.

The common movement is therefore not simply toward centralization.

Nor is it toward maximum abstraction.

It is toward consolidating what legitimately recurs while preserving separation where responsibility, authority or useful freedom still belongs locally.

That is the larger relationship reconstructed across the eight pressure points.

R2WX does not establish that this is how full-stack applications should be constructed. It records where this particular engineering exploration arrived, through a runnable implementation in which those decisions can be inspected rather than accepted only from their description.

R2WX started from ordinary full-stack application problems.

What remains is an attempt to make fewer solved problems become implementation work again—without making shared infrastructure responsible for decisions that still belong somewhere else.

Investigation Design

The preceding examination was not produced through a single analytical pass.

Its development passed through two different investigative levels and eventually used three deliberately distinct boundaries of responsibility: analysis, epistemic discipline and author consciousness.

The boundaries were permeable. Every participant could reason across them, object to a finding, introduce contradictory material, or request that an apparently settled result be examined again.

Their primary responsibilities nevertheless remained distinct.

Analysis — Gemini

Gemini held the principal responsibility for excavating the artifact.

Its investigation was conducted through deliberately constrained, largely faceless repository-analysis prompts rather than through participation in the ongoing interpretive discussion. It was required to inspect implementation material, follow relationships across the repository, retrieve supporting evidence and develop candidate explanations from what could actually be found.

Its responsibility was not to preserve an existing interpretation of R2WX.

It was to keep returning to the artifact.

Epistemic discipline — ChatGPT

ChatGPT held the principal responsibility for what the accumulated evidence permitted the investigation to conclude.

Implementation observations, author testimony and attractive interpretations were not treated as equivalent forms of evidence. Larger conclusions could be resisted when their evidential bridge remained incomplete, classifications could be narrowed when their terminology exceeded what had been established, and previously accepted explanations could be reopened when additional evidence changed their meaning.

Its responsibility was not to maximize the significance of the findings.

It was to prevent the claims from outrunning what had been established.

Consciousness — the author

The author's privileged position created a different responsibility.

Having constructed the artifact, the author could recognize when an explanation was locally correct while substantial engineering embodied elsewhere in the system remained unaccounted for.

That knowledge did not determine what the missing material would eventually be called or what larger conclusion it would support. It could instead require the investigation to continue: inspect further, connect apparently unrelated behavior, recover mechanisms that had not yet entered the reconstruction, and avoid treating a plausible partial explanation as completion.

The author's primary responsibility was therefore to prevent the investigation from closing while engineering embodied in the artifact remained undiscovered.

Re-examination across the boundaries

These responsibilities did not establish three isolated analytical territories.

Gemini could propose interpretations. ChatGPT could perform implementation analysis. The author could challenge classifications. Each participant could contribute within another participant's primary jurisdiction.

Likewise, every participant could object.

An objection did not determine the replacement conclusion.

It reopened the investigation.

Additional repository evidence could require the author to reconsider an earlier reconstruction. An epistemic objection could return a candidate interpretation to repository analysis. Author awareness of unexplored implementation could require both analytical positions to regenerate their explanations.

Re-examination therefore carried no predetermined direction.

A finding could become stronger.

It could become narrower.

Two apparently related findings could separate.

Apparently unrelated mechanisms could become connected.

A classification could change.

Or a claim could be rejected.

The responsibility boundaries determined what each participant was principally accountable for. They did not protect any participant's previous analysis from revision.

Two investigative levels

The triad was not the beginning of the investigation.

An earlier analytical level established the factual basis from which the later reconstruction could proceed.

Level 1 — Establishing the basis

The initial work combined author interrogation with original implementation artifacts.

The process advanced incrementally:

```
claim → artifact → interrogation → qualification → agreement → documented finding
```

This produced a series of reports in which author knowledge could be compared against concrete implementation evidence rather than accepted as sufficient by itself.

Those reports established substantial technical and historical ground.

They did not, however, constitute a complete reconstruction of R2WX as one engineering object. Their unit of progress was principally the individual claim, artifact or question being investigated.

Level 2 — Reconstructing the whole

The second level deliberately reopened the object.

A fresh analytical context was introduced. Gemini performed blind repository excavation. ChatGPT operated from a resistant epistemic position. The author moved from answering individual implementation questions toward requiring the investigation to account for the artifact as a whole.

Earlier findings remained available, but they were not immune to re-examination.

The objective was no longer merely to establish whether individual claims were supported.

It was to determine how the observed decisions related to one another across the complete system.

The eight pressure points used in this paper were not supplied as that investigation's original analytical framework.

They are part of its result.

Publication and repository exposure

A fourth participant, GitHub Copilot, entered temporarily under a narrower jurisdiction after the artifact and its engineering interpretation had substantially stabilized.

Its role concerned GitHub reality: repository presentation, discoverability, terminology, topic selection and indexing.

That work did not determine what R2WX was allowed to mean.

The engineering investigation determined the claims that could legitimately accompany the artifact; the GitHub-facing pass addressed how those claims and the repository could be exposed effectively within that environment.

Inspecting the investigation

The methodology described here is not offered as proof that the resulting interpretation is correct.

It records how the interpretation was produced.

Representative prompts, evidence classifications and selected instances in which findings were reopened, narrowed, rejected or reorganized can accompany this section so that the analytical process is itself available for inspection.

The same distinction used throughout the R2WX examination applies here:

the procedure can be exposed; its value remains for the reader to judge.